TECHNICAL DOCUMENTATION

MoM Agent

A full-stack AI pipeline that transforms meeting recordings into structured Minutes of Meeting documents using Deepgram and Claude.

Version
1.0

Model
claude-sonnet-4-6

Stack
Next.js · FastAPI · Deepgram · Anthropic

Hosting
Vercel · Render

Table of Contents

1 Overall Architecture

2 Data Flow

3 Tech Stack

4 Core Modules & Responsibilities

- 4.1 backend/main.py
- 4.2 backend/api/routes/meeting.py
- 4.3 backend/services/transcriber.py
- 4.4 backend/services/mom_generator.py
- 4.5 backend/services/audio_extractor.py
- 4.6 backend/services/file_handler.py
- 4.7 frontend/lib/api.ts
- 4.8 frontend/app/page.tsx

5 Key Features & Logic

6 External Integrations

7 Execution Flow (Step-by-Step)

8 Edge Case Handling

9 Assumptions & Limitations

1 Overall Architecture

The MoM Agent is a full-stack web application with a clear separation between the **Next.js frontend** and the **FastAPI backend**. The system accepts a meeting recording, processes it through a multi-stage AI pipeline, and returns a structured Minutes of Meeting document.

High-Level Component Diagram

```
FRONTEND (Next.js / Vercel)
  page.tsx ↔ lib/api.ts ↔ NEXT_PUBLIC_API_URL
          │ HTTP REST
BACKEND (FastAPI / Render)
  main.py
  └── api/routes/meeting.py (3 endpoints)
       ├── services/file_handler.py
       ├── services/audio_extractor.py ──→ ffmpeg
       ├── services/transcriber.py ──→ Deepgram API
       └── services/mom_generator.py ──→ Claude API (3 calls)
```

Key Design Decisions

ASYNC UPLOAD

The upload endpoint returns `202` immediately. All heavy processing runs in a `BackgroundTask`, preventing HTTP timeout on Render's free tier.

IN-MEMORY STATE

All meeting state is stored in a Python `dict` (meetings). No database — state is lost on every server restart.

POLLING MODEL

The frontend polls `GET /meeting/{id}/status` every 3 seconds until status is `"completed"` or `"failed"`.

3-CALL CLAUDE PIPELINE

The MoM pipeline makes 3 separate Claude calls for speaker resolution, attendee extraction, and MoM generation.

2 Data Flow

Stage 1 — Frontend to Backend

1. `wakeUpBackend()` pings `GET /health` every 5 seconds for up to 2 minutes (handles Render cold starts)
2. `POST /meeting/create` — sends `meeting_name` + `meeting_date` — receives `meeting_id`
3. `POST /meeting/{id}/upload` — sends audio/video file as `multipart/form-data` — receives `202` immediately

Stage 2 — Backend Background Processing

1. `file_handler.save_upload()` writes to `tmp/uploads/<uuid>.<ext>`
2. `audio_extractor.extract_audio()` runs ffmpeg → outputs 64 kbps mono 16 kHz MP3 to `tmp/audio/<uuid>.mp3`
3. `transcriber.transcribe()` calls Deepgram Nova-2 API → returns `(plain_transcript, speaker_transcript)`
4. `mom_generator.generate_mom(speaker_transcript)` runs the 3-step Claude pipeline → structured JSON
5. `cleanup_files()` always runs in the `finally` block, deletes both temp files

Stage 3 — Frontend Polling & Render

1. Frontend polls `GET /meeting/{id}/status` every 3 seconds
2. On `"completed"`: renders MoM document and raw transcript tabs
3. On `"failed"`: shows error message with backend reason

Transcript Data Split

Field	Source	Usage
`plain_transcript`	Deepgram channel alternative	Shown in the Transcript tab
`speaker_transcript`	Deepgram utterances	Passed to MoM generator for name resolution

3 Tech Stack

Layer	Technology	Purpose
Frontend Framework	Next.js 16 (React 19)	UI, routing, API proxying
Frontend Styling	Tailwind CSS 4 + CSS variables	Design system, responsive layout
Notifications	Sonner	Toast messages
Icons	Lucide React	UI icons
Backend Framework	FastAPI	REST API, background tasks, CORS
Backend Server	Uvicorn	ASGI server
Audio Processing	ffmpeg-python	Audio extraction and compression
Transcription	Deepgram Nova-2	Speech-to-text with speaker diarization
MoM Generation	Claude claude-sonnet-4-6 (Anthropic)	3-step NLP pipeline
HTTP Client	requests	Deepgram API calls from backend
File I/O	aiofiles	Async file write for uploads
Env Management	python-dotenv	.env loading
Backend Hosting	Render (free tier)	Cloud hosting for FastAPI
Frontend Hosting	Vercel	Edge-hosted Next.js

4 Core Modules & Responsibilities

backend/main.py

Creates the FastAPI app instance and mounts the meeting router.

CORS config | Global exception handler | localhost:3000 | *.onrender.com | *.vercel.app

Adds a **global exception handler** that attaches CORS headers to every response including 500 errors, so CORS errors never mask actual server errors. On startup, logs whether `OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, and `DEEPGRAM_API_KEY` are present.

backend/api/routes/meeting.py

The only API layer. Owns the meetings in-memory dictionary.

Endpoint	Method	Responsibility
`/meeting/create`	POST	Stores `meeting_name`, `meeting_date`. Sets status to `"pending"`.
`/meeting/{id}/upload`	POST	Validates file type, saves file, triggers `BackgroundTask`. Returns `202`.
`/meeting/{id}/status`	GET	Returns current status, transcript, and MoM JSON.

The `process_audio_task()` background function progresses through statuses: `extracting → transcribing → generating → completed` (or `failed` on error).

backend/services/transcriber.py

Reads compressed MP3 bytes and POSTs them to the Deepgram Nova-2 API.

KEY DEEPGRAM PARAMETERS

`model=nova-2` · `diarize=true` · `smart_format=true` · `punctuate=true` · `utterances=true` · `filler_words=false`

Builds `plain_transcript` from `results.channels[0].alternatives[0].transcript`. Builds `speaker_transcript` from `results.utterances`, each line formatted as `"Speaker N: text"`. Returns both as a tuple.

backend/services/mom_generator.py

The most complex module. Runs a three-step Claude pipeline.

Function	Role
`_resolve_speaker_names()`	Step 1: maps Speaker IDs to real names via 3 explicit reasoning rules
`_extract_directly_addressed_names()`	Step 2: fallback extraction of confirmed attendees from direct address patterns
`_apply_speaker_names()`	regex-replaces "Speaker N:" labels with resolved names in the transcript
`_strip_speaker_labels()`	post-processing safety net: recursively removes surviving "Speaker N" from final MoM dict
`generate_mom()`	orchestrates all steps and returns the final MoM dict

backend/services/audio_extractor.py

Uses `ffmpeg-python` to extract audio from video files or recompress audio files.

> **OUTPUT SPEC**
> 64 kbps · Mono · 16 kHz · MP3 — optimised for Deepgram transcription accuracy and bandwidth.

Runs synchronously inside the background task thread.

backend/services/file_handler.py

Function	Description
`save_upload()`	Writes multipart upload to `tmp/uploads/<uuid>.<ext>` using aiofiles (async)
`cleanup_files()`	Deletes temp files; always called in the `finally` block
`is_video()` / `is_audio()`	Validates extension against allowed lists

Audio: .mp3 .wav .m4a Video: .mp4 .mov .mkv

frontend/lib/api.ts

Centralises all HTTP calls. Reads `NEXT_PUBLIC_API_URL` (falls back to `http://localhost:8000`).

Function	Description
`wakeUpBackend()`	Polls `GET /health` every 5s for up to 2 minutes
`createMeeting()`	POST `/meeting/create` with meeting name and date
`uploadRecording()`	POST `/meeting/{id}/upload` as multipart/form-data
`getMeetingStatus()`	GET `/meeting/{id}/status`, polled every 3 seconds until done

frontend/app/page.tsx

Single-page React component managing all UI state.

waking · uploading · extracting · transcribing · generating · done

MoM tab renders attendees as chips, discussion points as a numbered list, and action items grouped under **Client Team / Spikra Team**. The Transcript tab shows raw plain text from Deepgram. The Copy button formats and copies MoM as markdown-style plain text to the clipboard.

5 Key Features & Logic

Speaker Resolution Pipeline

The three-step pipeline transforms `"Speaker N"` labels into real names.

Step 1 — Speaker Name Resolution

Claude analyses the diarized transcript using three ordered rules:

RULE 1 — HIGHEST PRIORITY

Self-Identification

Pattern: "I'm [Name]", "My name is [Name]"

```
Speaker 2: I'm Charan
→ {"2": "Charan"}
```

RULE 2

Direct Address + Response

Speaker A addresses [Name] and Speaker B responds

```
Speaker 0: Shini, explain this.
Speaker 1: Sure...
→ {"1": "Shini"}
```

RULE 3

Post-Speech Acknowledgment

"Thanks [Name]", "Good point [Name]" after B spoke

```
Speaker 0: Good point, Aditya.
→ previous speaker = Aditya
```

Step 2 — Confirmed Attendees Extraction

Sends the original unresolved transcript to Claude. Finds every name directly spoken to — greetings, acknowledgments, direct questions. Acts as fallback for names that couldn't be mapped to a speaker ID in Step 1.

Step 3 — MoM Generation

Input: name-resolved transcript + confirmed attendees list. Claude returns structured JSON with meeting metadata, attendees, discussion points, and action items. Action items split into exactly two teams: **Client Team** and **Spikra Team**.

Post-processing — `_strip_speaker_labels()`

Regex safety net replaces any surviving `"Speaker N"` in the output with `"a participant"`. Drops attendee entries that resolved to `"a participant"`. If no valid attendees remain, sets `["Not clearly identifiable from transcript"]`.

Attendee Extraction Rules

Rule	Detail
Only explicit names	Only names present in the transcript — as speaker label or directly addressed
No invention	NEVER invent, infer, or guess names
No raw labels	NEVER include "Speaker N" labels in output
Skip unclear fragments	"screen sir", "sni sir", "sir", "madam" — all filtered out
Normalise spelling	Include the person once; apply normalisation table
Greetings count	"Hi Aditya", "Hello Mina", "Thanks Sri" are valid extractions
Fallback	Fewer than 2 identifiable names → `["Not clearly identifiable from transcript"]`

Name Normalisation Table

Transcription Error	Corrected To
Aditiya	Aditya
Minna	Mina
Meena	Mina
Sree	Sri

Normalisation is applied only when the intended name is certain. Uncertain cases are left as `null` or omitted.

6 External Integrations

Deepgram Nova-2 (Transcription)

Property	Value
Endpoint	`POST https://api.deepgram.com/v1/listen`
Auth	`Authorization: Token DEEPGRAM_API_KEY`
Input	Raw MP3 bytes in request body
Output	JSON with `results.channels` and `results.utterances`
Free Tier	200 hours/month
Typical Latency	5–15 seconds for a 1-hour meeting

Anthropic Claude claude-sonnet-4-6 (MoM Generation)

Property	Value
SDK	`anthropic` Python package
Auth	`ANTHROPIC_API_KEY`
Calls per meeting	3 (speaker resolution, attendee extraction, MoM generation)
Max tokens	4096 per call
Temperature	0.0 for Steps 1 & 2 (deterministic), 0.1 for Step 3

Environment Variables

`DEEPGRAM_API_KEY`
Used by transcriber.py for the Deepgram API

`ANTHROPIC_API_KEY`
Used by mom_generator.py for the 3-step Claude pipeline

`OPENAI_API_KEY`
Retained for rollback in main.py — not actively used

`NEXT_PUBLIC_API_URL`
Backend URL for the Next.js frontend (api.ts)

7 Execution Flow (Step-by-Step)

1. User opens the app and fills in **Meeting Name**, **Meeting Date**, and selects a file.
2. `wakeUpBackend()` pings `GET /health` every 5 seconds for up to 2 minutes to handle Render cold starts.
3. `createMeeting()` calls `POST /meeting/create` — backend stores meeting metadata and returns `meeting_id`.
4. `uploadRecording()` calls `POST /meeting/{id}/upload` — backend validates file type, saves to `tmp/uploads/`, returns `202` immediately.

5a. **BackgroundTask:** `extract_audio()` — ffmpeg compresses to 64 kbps mono 16 kHz MP3.

5b. **BackgroundTask:** `transcribe()` — Deepgram Nova-2 returns plain and speaker-diarized transcripts in 5–15 seconds.

5c. **BackgroundTask:** `generate_mom()` Step 1 — Claude resolves Speaker IDs to names.

5d. **BackgroundTask:** `generate_mom()` Step 2 — Claude extracts confirmed attendees via direct address patterns.

5e. **BackgroundTask:** `generate_mom()` Step 3 — Claude generates structured MoM JSON. `_strip_speaker_labels()` post-processes the output. `meetings[id]` updated to `status="completed"`. `cleanup_files()` deletes temp files.

6. Frontend receives `"completed"` on next poll. Renders the MoM document and transcript tabs.
7. User can copy the MoM to clipboard or view the raw transcript.

8 Edge Case Handling

Scenario	Handling
Speaker name cannot be resolved	Step 1 returns null. Step 2 tries via direct address. `_strip_speaker_labels()` catches any remaining labels.
Unclear transcription ("screen sir")	Explicitly filtered in all three Claude prompts and the MoM system prompt.
Name spelling variants ("Aditiya")	Normalisation table applied in all prompts. Claude corrects only when certain.
Fewer than 2 identifiable attendees	`attendees` set to `["Not clearly identifiable from transcript"]`.
Claude returns markdown-wrapped JSON	`_parse_json_response()` strips ```` ```json ```` fences before parsing.
Claude returns invalid JSON	`generate_mom()` raises `RuntimeError`. Background task sets `status = "failed"`.
Render backend cold start (up to 50s)	`wakeUpBackend()` retries for 2 minutes before timing out.
Video file uploaded instead of audio	`audio_extractor.py` detects and strips the audio stream via ffmpeg.
Temp files left after crash	`cleanup_files()` is in a `finally` block — always runs even on exception.
CORS error on server failures	`main.py` adds CORS headers in a global exception handler so 500 errors don't appear as CORS errors in the browser.

9 Assumptions & Limitations

Assumptions

LANGUAGE

Meetings are in English. Deepgram Nova-2 supports other languages but the Claude prompts and normalisation rules are English-only.

AUDIO QUALITY

Speaker diarization quality depends on audio clarity — overlapping speech and background noise reduce accuracy.

NAME ADDRESS PATTERNS

Attendee name resolution assumes speakers address each other by name at some point in the meeting. If they never do, speaker IDs remain unresolved.

Known Limitations

- ⚠ **In-memory state only** — All meeting data lost on every server restart. Render free tier restarts frequently.
- ⚠ **No authentication** — Anyone with the backend URL can create meetings and upload files.
- ⚠ **Sequential Claude calls** — Steps 1, 2, and 3 run one after another. Total LLM latency is approximately 15–30 seconds.
- ⚠ **Speaker resolution is probabilistic** — If the transcript lacks address patterns, Step 1 returns null for that speaker.
- ⚠ **No persistent file storage** — Temp files deleted after processing. Meetings cannot be reprocessed.
- ⚠ **Single backend instance** — The `meetings` dict is not shared across instances. Would break if scaled horizontally.
- ⚠ **Long meetings** — Very long recordings (3+ hours) may produce large files even at 64 kbps.
- ⚠ **Render cold starts** — Backend can take up to 50 seconds to wake up. Frontend waits up to 2 minutes.
- ⚠ **OPENAI_API_KEY loaded but unused** — A leftover from the prior GPT-4o implementation. Safe to remove if rollback is no longer needed.